Parallel and Distributed Computing Overview

(CS 3006)

Muhammad Aadil Ur Rehman

Department of Computer Science,
National University of Computer & Emerging Sciences,
Islamabad Campus

Credits: Dr. Muhammad Aleem

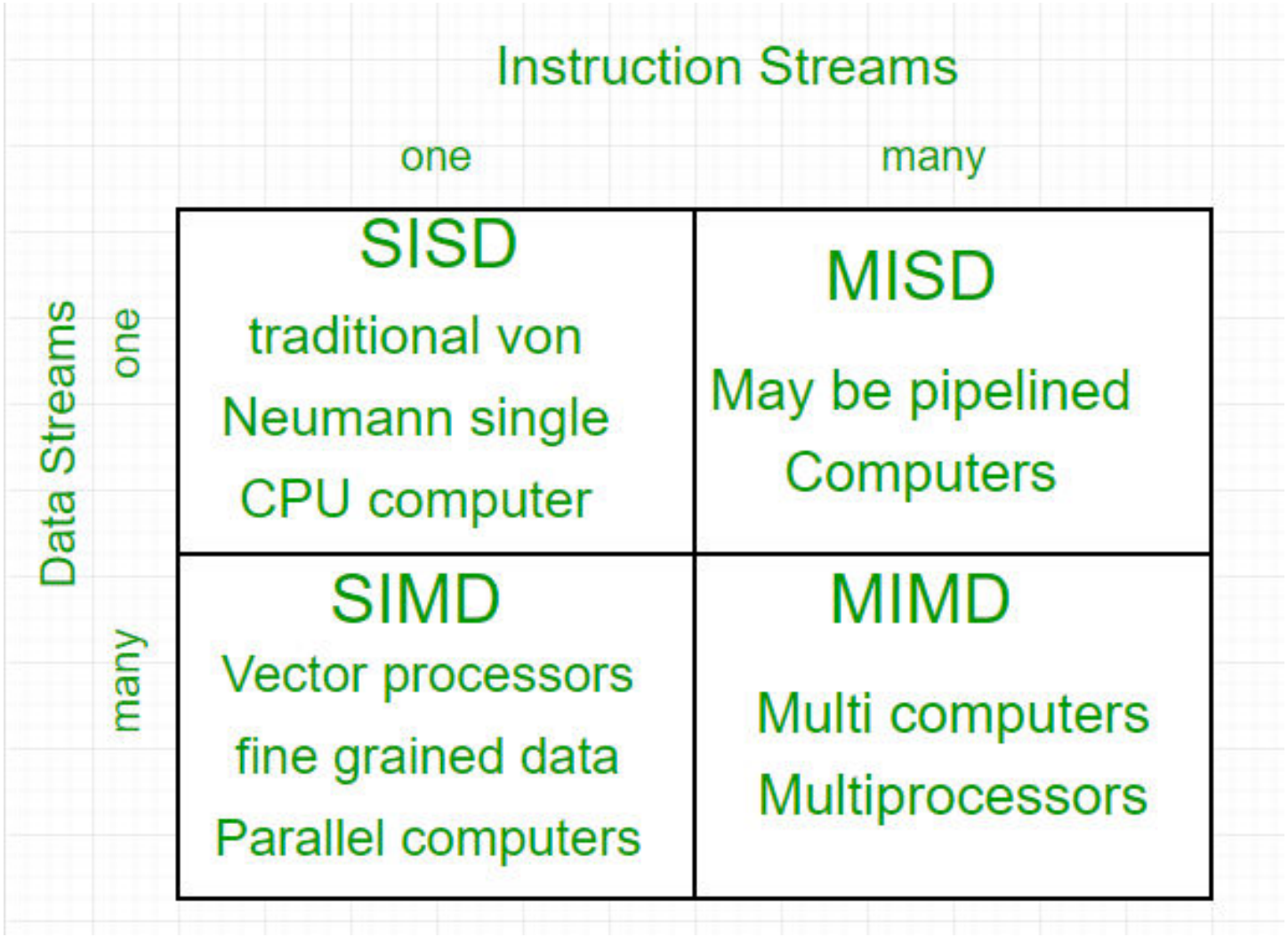
Flynn's Taxonomy

- Specific **classification** of **parallel architecture**

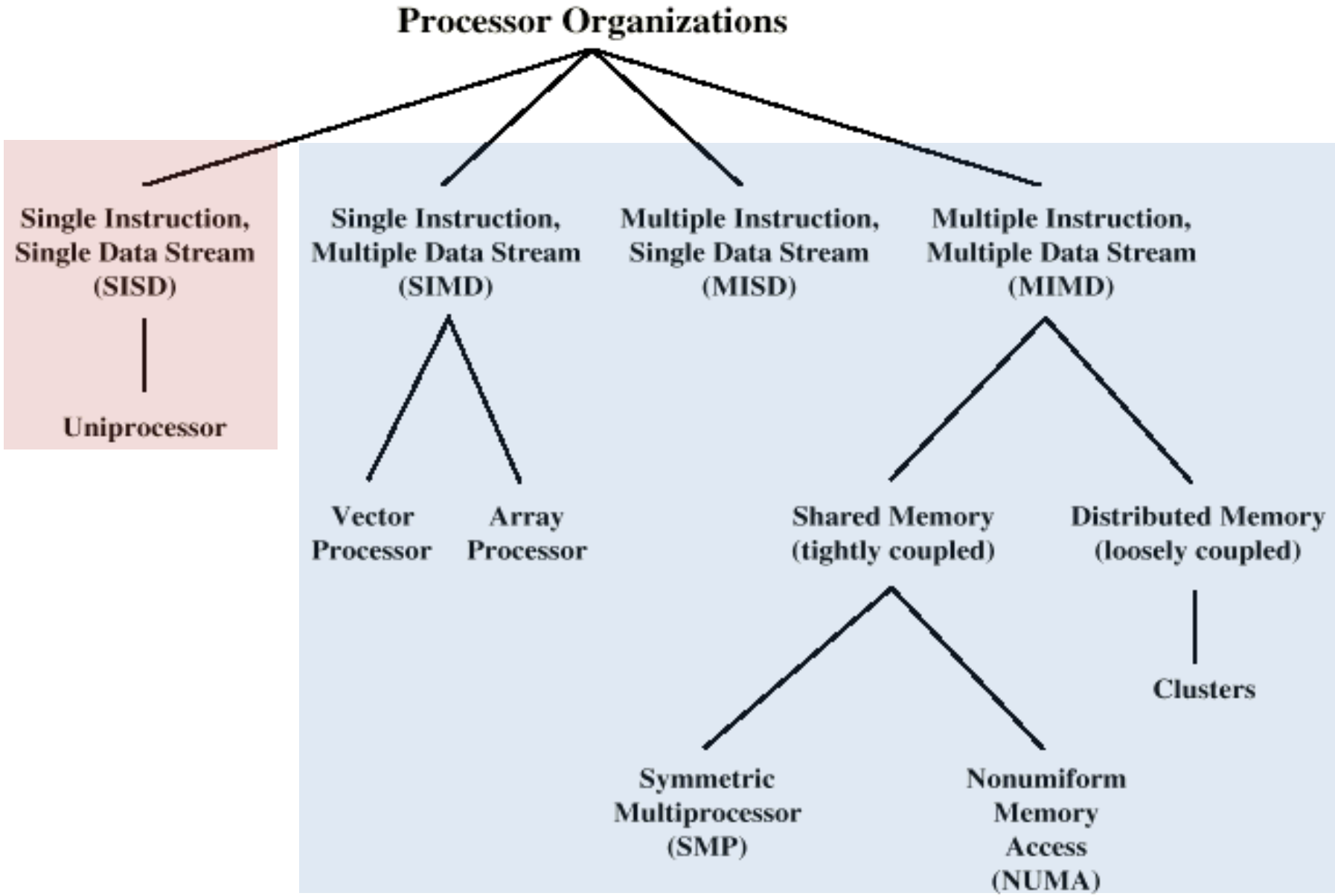
- By **Michael Flynn** (from Stanford, in 1966)
  - Made a **classification** of **computer systems** known as **Flynn's Taxonomy**

- Computer
  - Instructions
  - Data

Flynn's Taxonomy

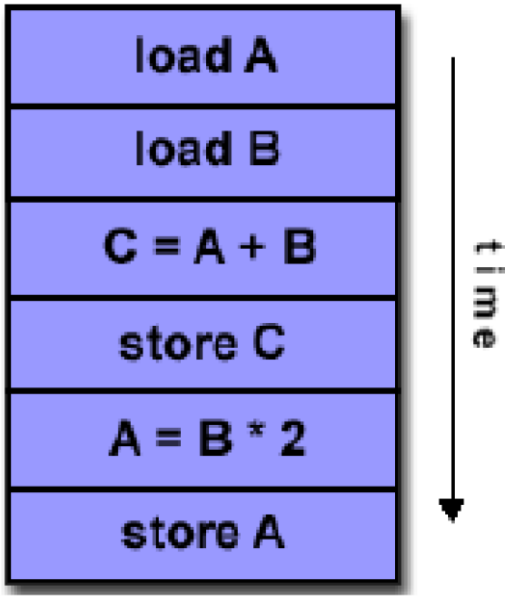


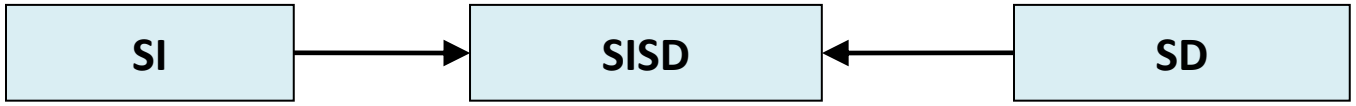
Taxonomy of Processor Architectures



1. Single Instruction, Single Data Stream – *SISD*

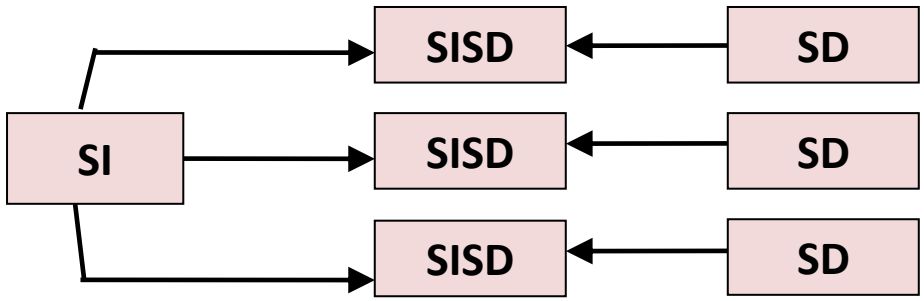
- Single processor
- Single instruction stream
- Data stored in single memory
- Deterministic execution





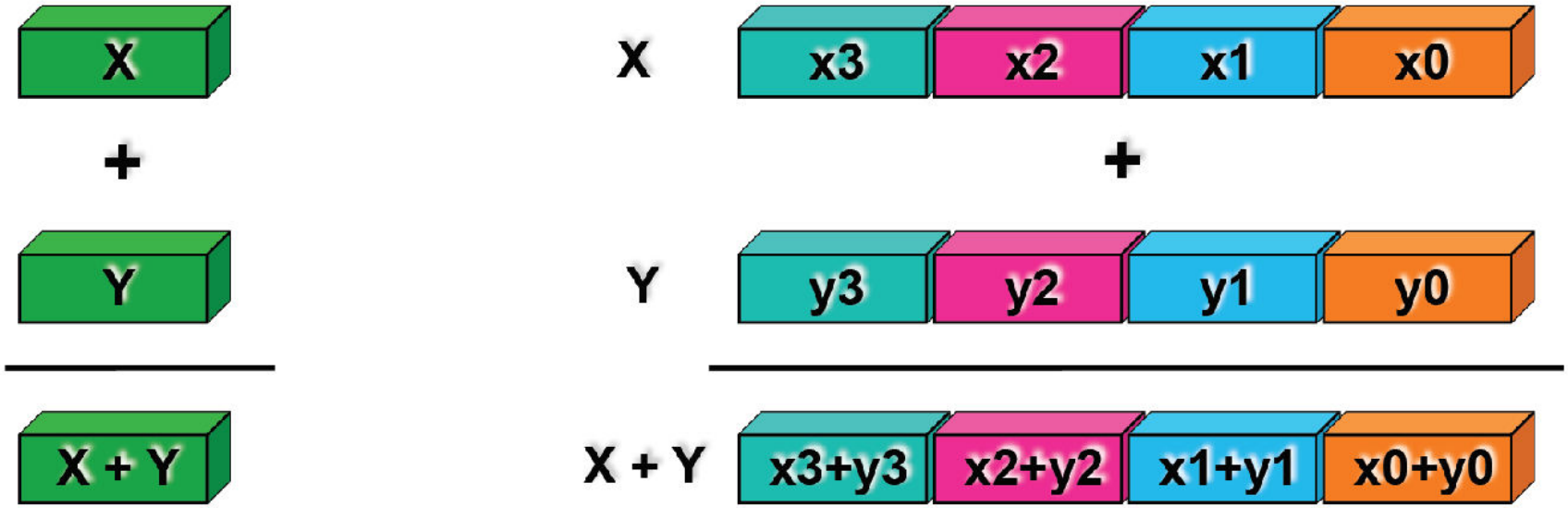
2. Single Instruction, Multiple Data Stream - *SIMD*

- A **parallel processor**
- **Single instruction**: all **processing units** **execute same instruction** at **any given clock cycle**
- **Multiple data**: Each **processing unit** can **operate on** a **different data element**
- **Large number** of **processing elements** **(with local memory)**
- **Examples**: **GPUs, etc.**



2. Single Instruction, Multiple Data Stream - *SIMD*

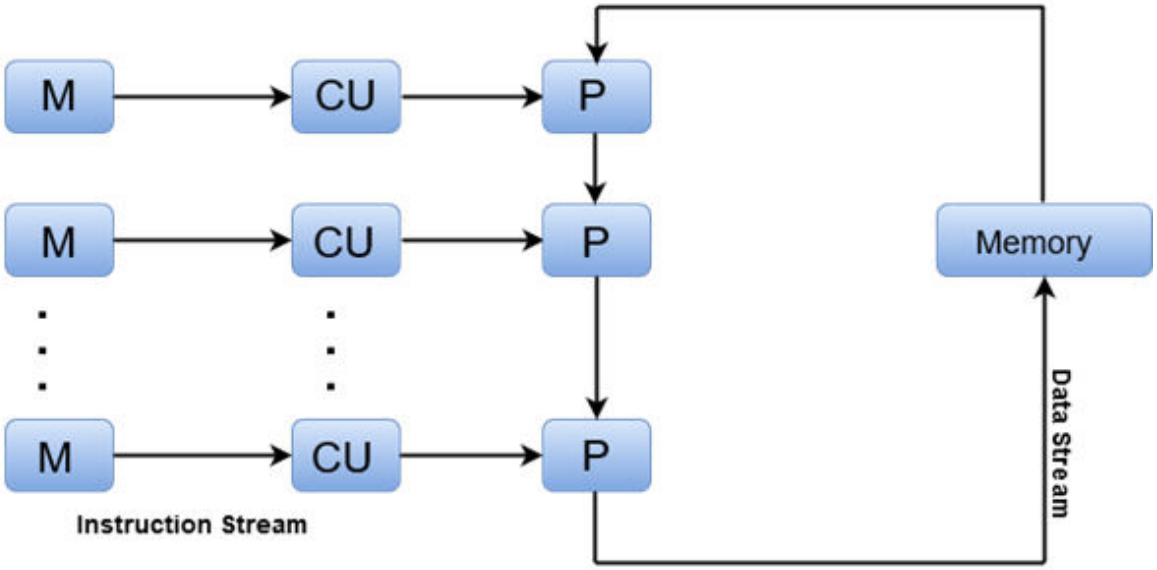
- **Scalar processing**
  - —one operation produces one result
- **SIMD vector units**
  - —one operation produces multiple results



Slide Credit: Alex Klimovitski & Dean Macri, Intel Corporation

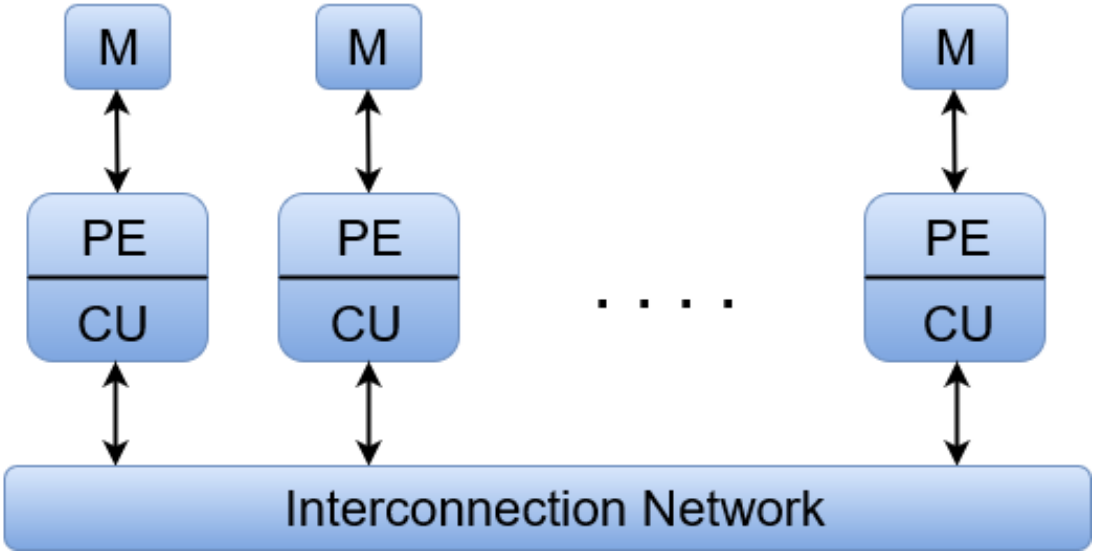
3. Multiple Instruction, Single Data Stream - *MISD*

- **Sequence** of **data**
- Transmitted to **set of processors**
- **Each processor executes different instruction** sequence, using **same Data**
- Examples: *Pipelined Vector Processors, etc.*



4. Multiple Instruction, Multiple Data Stream- *MIMD*

- **Most common parallel processor architecture**
- **Simultaneously execute different instructions**
- Using **different** sets of **data**
- Examples: ***Multi-cores***, ***SMPs***, ***Clusters, Grid, Cloud***

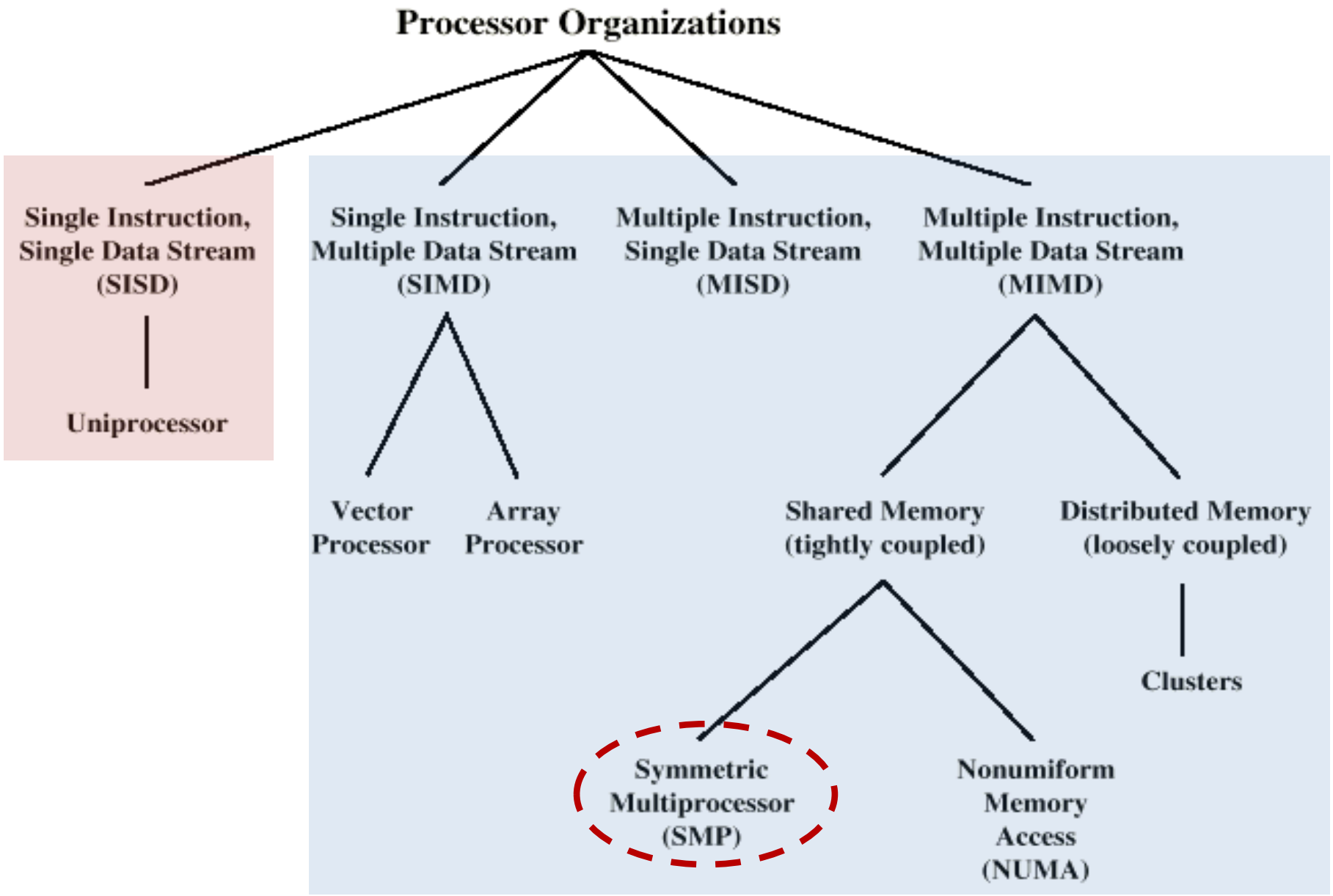


MIMD - Overview

- **General purpose processors**
- **Each** can **process** all **instructions** necessary
- Further ***classified by method of processor communication:***

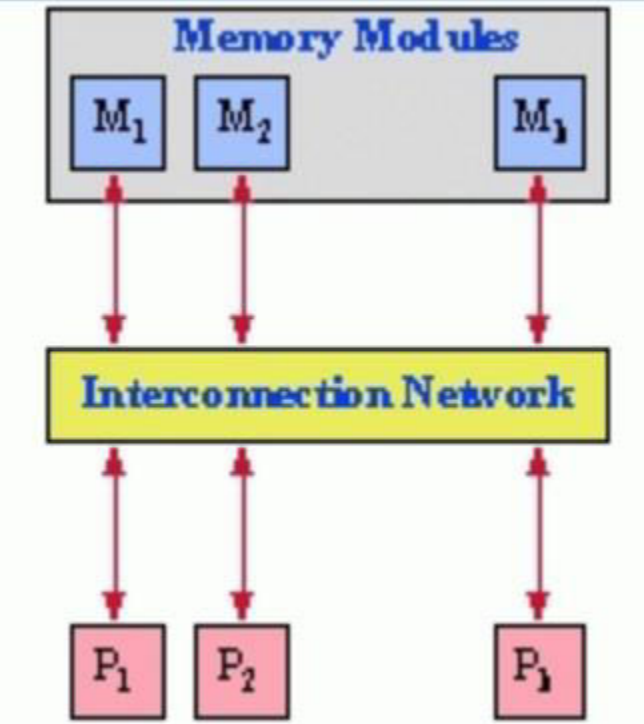
    1. **Shared Memory**
    2. **Distributed Memory (Will be discussed later)**

Taxonomy of Processor Architectures

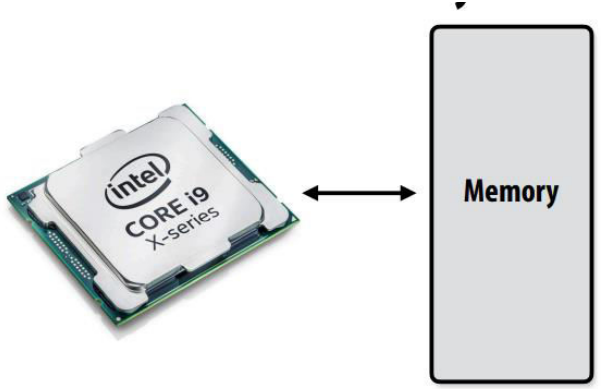


Symmetric Multiprocessor (SMP)

- Processors share memory (tightly coupled)
- Communicate via shared memory (single bus)
- Same memory access time (any memory region, from any processor)
  - Uniform Memory Access (UMA)
- Processors share I/O address space too



Accessing Mem



A program's memory address space

- A computer's memory is organized as an array of bytes

- Each byte is identified by its "address" in memory (its position in this array)
  (We'll assume memory is byte-addressable)

"The byte stored at address 0x8 has the value 32."

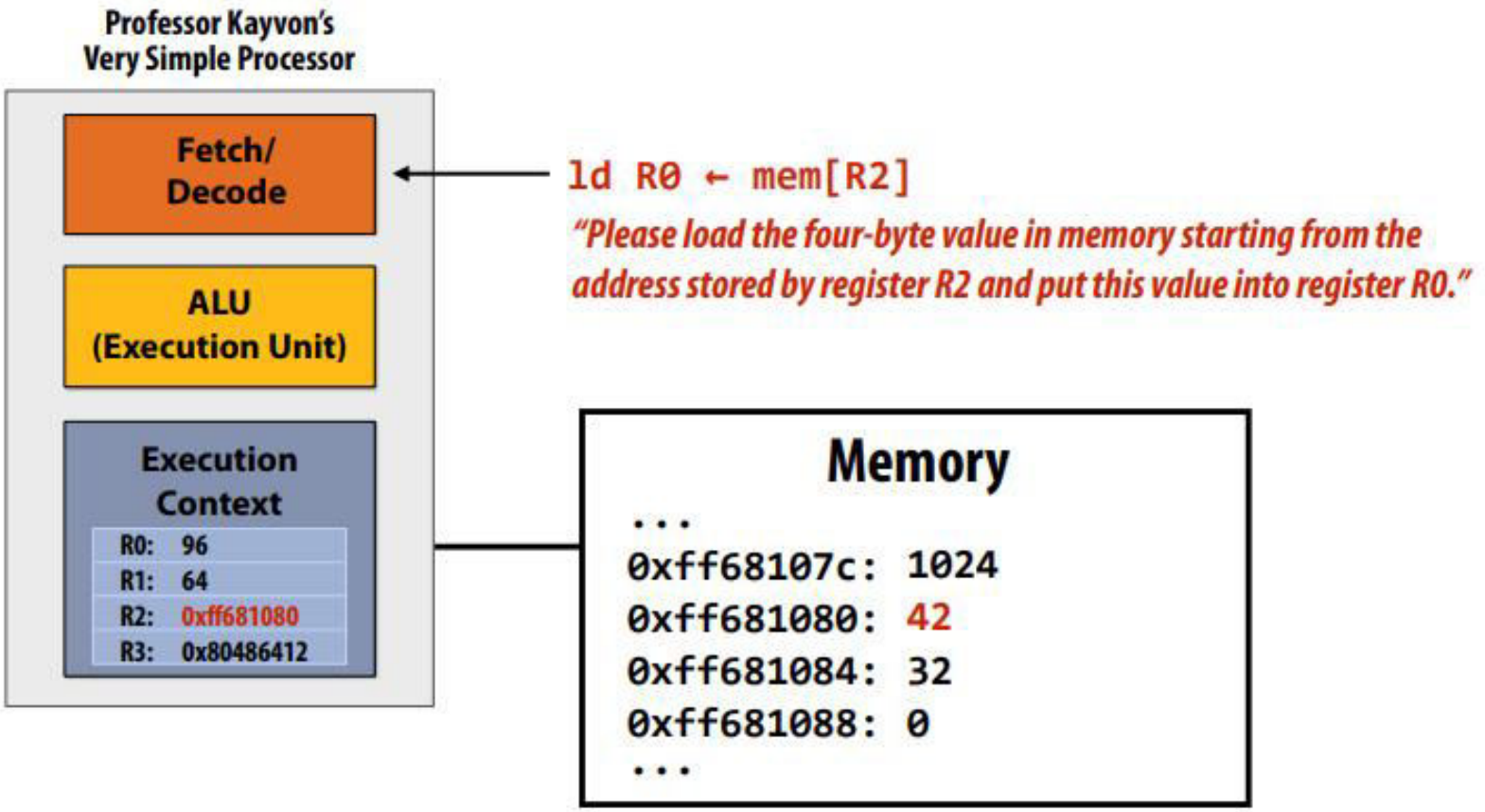
"The byte stored at address 0x10 (16) has the value 128."

In the illustration on the right, the program's memory address space is 32 bytes in size (so valid addresses range from 0x0 to 0x1F)

Address	Value
0x0	16
0x1	255
0x2	14
0x3	0
0x4	0
0x5	0
0x6	6
0x7	0
0x8	32
0x9	48
0xA	255
0xB	255
0xC	255
0xD	0
0xE	0
0xF	0
0x10	128
⋮	⋮
0x1F	0

Accessing Memory

Load: an instruction for accessing the contents of memory



Review from Lecture 1

- **Memory access latency**
  - The amount of time it takes the memory system to provide data to the processor
  - Example: 100 clock cycles, 100 nsec

- A processor "stalls" (can't make progress) when it cannot run the next instruction in an instruction stream because future instructions depend on a previous instruction that is not yet complete.

- **Accessing memory is a major source of stalls**

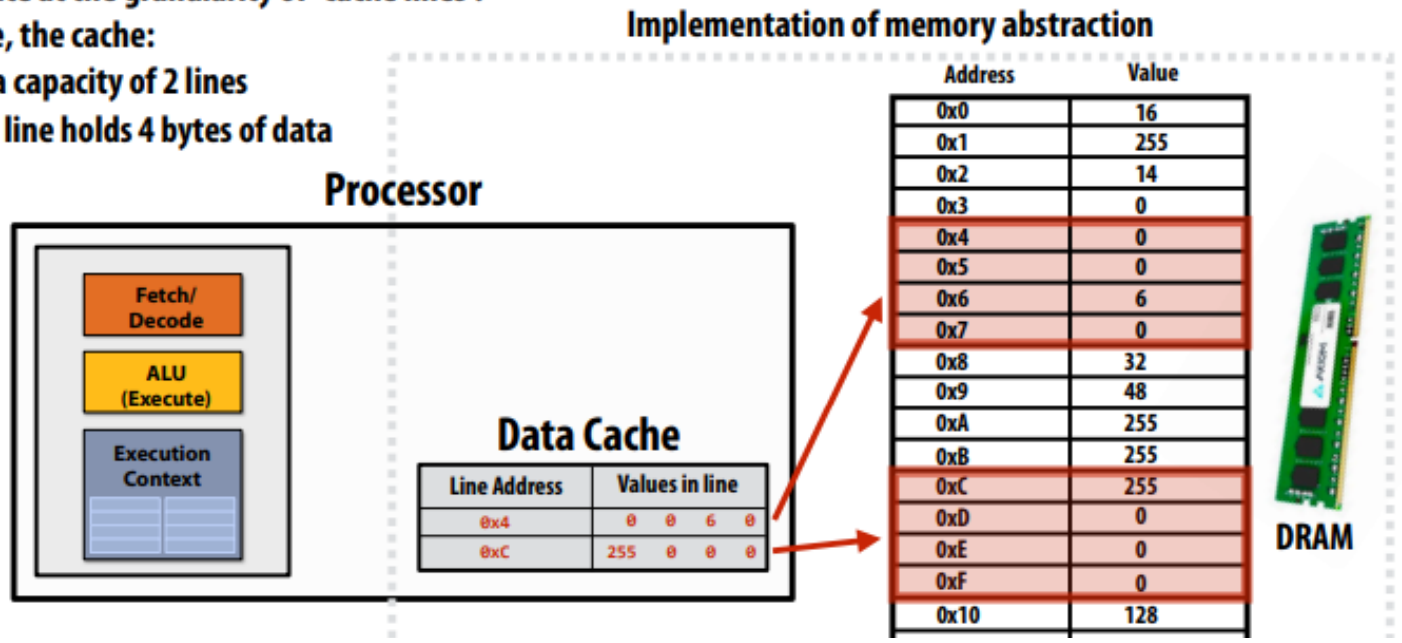
```
ld r0 mem[r2]
ld r1 mem[r3]
add r0, r0, r1
```

Dependency: cannot execute 'add' instruction until data from mem[r2] and mem[r3] have been loaded from memory

What Are Caches?

- A cache is a hardware implementation detail that does not impact the output of a program, only its performance
- Cache is on-chip storage that maintains a copy of a subset of the values in memory
- If an address is stored "in the cache" the processor can load/store to this address more quickly than if the data resides only in DRAM
- Caches operate at the granularity of "cache lines".

In the figure, the cache:
- Has a capacity of 2 lines
- Each line holds 4 bytes of data



How does a processor decide what data to keep in cache?

- Outside the scope of this course, but I suggest googling these terms…
  - Direct mapped cache
  - Set-associative cache
  - Cache line

Cache example 1

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	48
	0xA	255
	0xB	255
Line 0xC	0xC	255
	0xD	0
	0xE	0
	0xF	0

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

time

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0 ●●●●
0x1	hit	0x0 ●●●●
0x2	hit	0x0 ●●●●
0x3	hit	0x0 ●●●●
0x2	hit	0x0 ●●●●
0x1	hit	0x0 ●●●●
0x4	"cold miss", load 0x4	0x0 ●●●● 0x4 ●●●●
0x1	hit	0x0 ●●●● 0x4 ●●●●

There are two forms of "data locality" in this sequence:

Spatial locality: loading data in a cache line "preloads" the data needed for subsequent accesses to different addresses in the same line, leading to cache hits

Temporal locality: repeated accesses to the same address result in hits.

Cache example 2

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	48
	0xA	255
	0xB	255
Line 0xC	0xC	255
	0xD	0
	0xE	0
	0xF	0

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

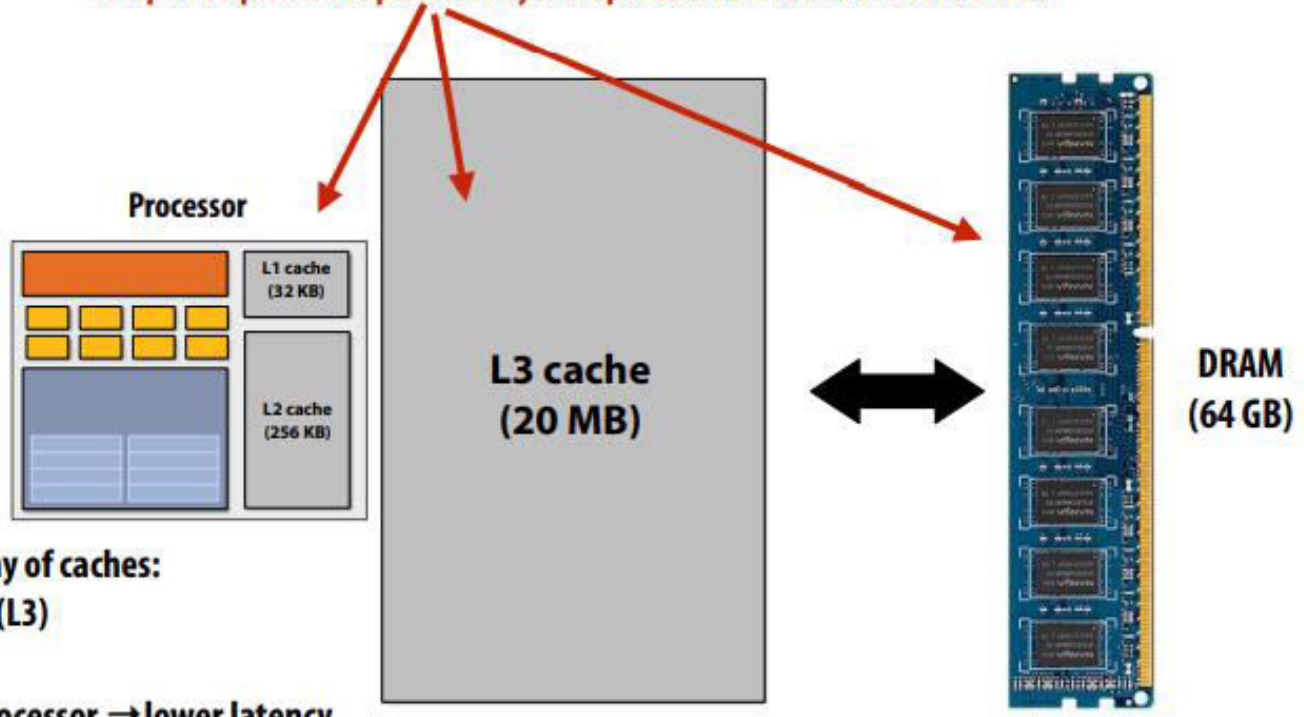
time

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0
0x1	hit	0x0
0x2	hit	0x0
0x3	hit	0x0
0x4	"cold miss", load 0x4	0x0 0x4
0x5	hit	0x0 0x4
0x6	hit	0x0 0x4
0x7	hit	0x0 0x4
0x8	"cold miss", load 0x8 (evict 0x0)	0x8 0x4
0x9	hit	0x8 0x4
0xA	hit	0x8 0x4
0xB	hit	0x8 0x4
0xC	"cold miss", load 0xC (evict 0x4)	0x8 0xC
0xD	hit	0x8 0xC
0xE	hit	0x8 0xC
0xF	hit	0x8 0xC
0x0	"capacity miss", load 0x0 (evict 0x8)	0x0 0xC

- **Processors run efficiently when they access data that is resident in caches**
- **Caches reduce memory access latency when processors accesses data that they have recently accessed! ***

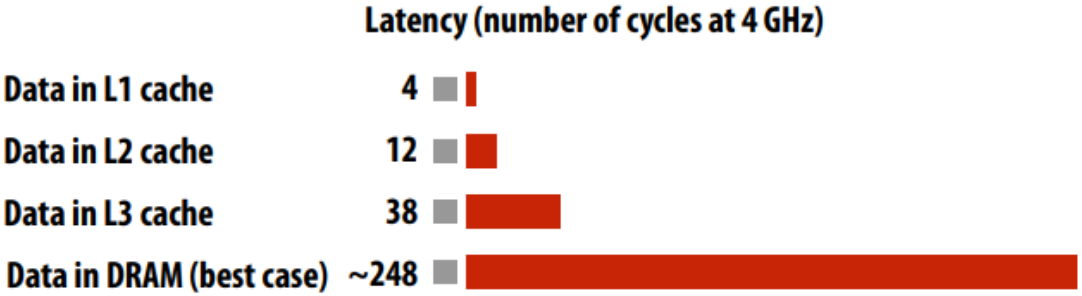
The implementation of the linear memory address space abstraction on a modern computer is complex

The instruction "load the value stored at address X into register R0" might involve a complex sequence of operations by multiple data caches and access to DRAM

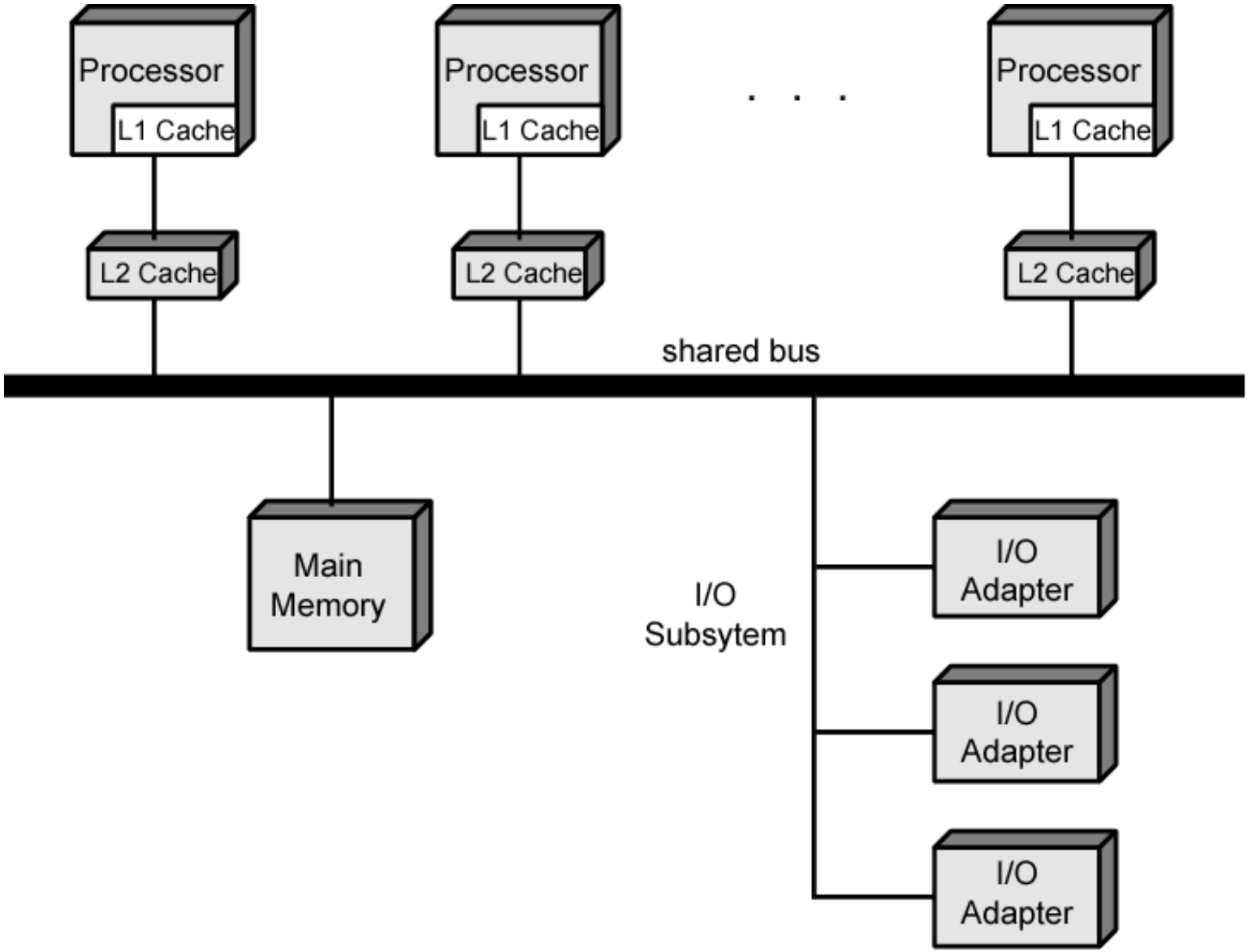


Common organization: hierarchy of caches:
Level 1 (L1), level 2 (L2), level 3 (L3)

Smaller capacity caches near processor → lower latency
Larger capacity caches farther away → larger latency



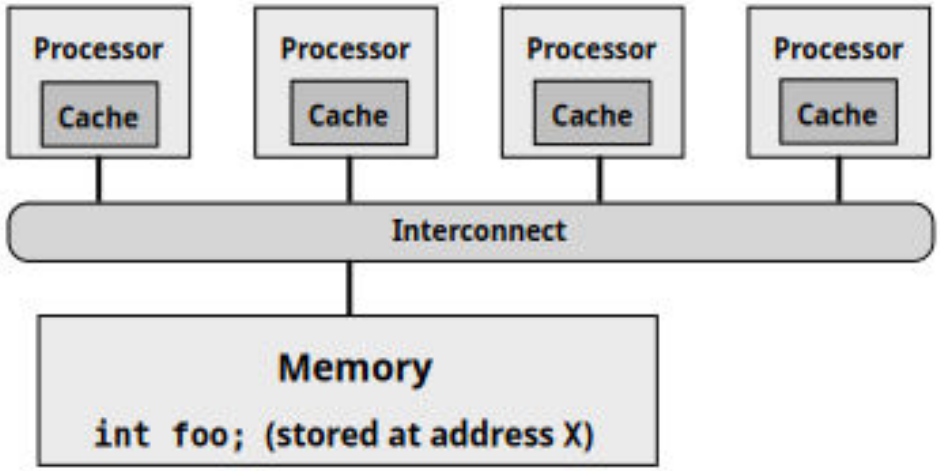
Symmetric Multiprocessor Organization



Cache Coherence Problem

Modern processors replicate contents of memory in local caches

Problem: processors can observe different values for the same memory location



Is this a mutual exclusion problem?

I.e., can you fix the problem by adding locks to your program?

NO!

This is a problem created by replicating the data stored at address X in local caches (a hardware implementation detail)

The chart at right shows the value of variable foo (stored at address X) in main memory and in each processor's cache

Assume the initial value stored at address X is 0

Assume write-back cache behavior

Action	P1 $	P2 $	P3 $	P4 $	mem[X]
					0
P1 load X	0 miss				0
P2 load X	0	0 miss			0
P1 store X	1	0			0
P3 load X	1	0	0 miss		0
P3 store X	1	0	2		0
P2 load X	1	0 hit	2		0
P1 load Y (assume this load causes eviction of X from P1 Cache)		0	2		

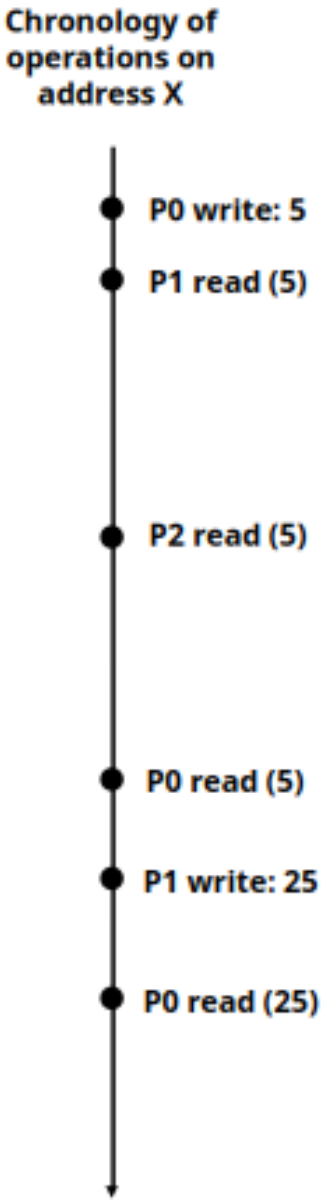
Cache Coherence Problem

A memory system is coherent if:

The results of a parallel program's execution are such that for each individual memory location, there is a hypothetical serial order of all program operations (executed by all processors) to the location that is consistent with the results of execution, and:

1. **Memory operations issued by any one processor occur in the order issued by the processor**
2. **The value returned by a read is the value written by the last write to the location... as given by the serial order**

Also known as *sequential consistency*



- **Cache Coherence in SMPs is maintained by Hardware, so they are also known as CC-UMA**

SMP Advantages

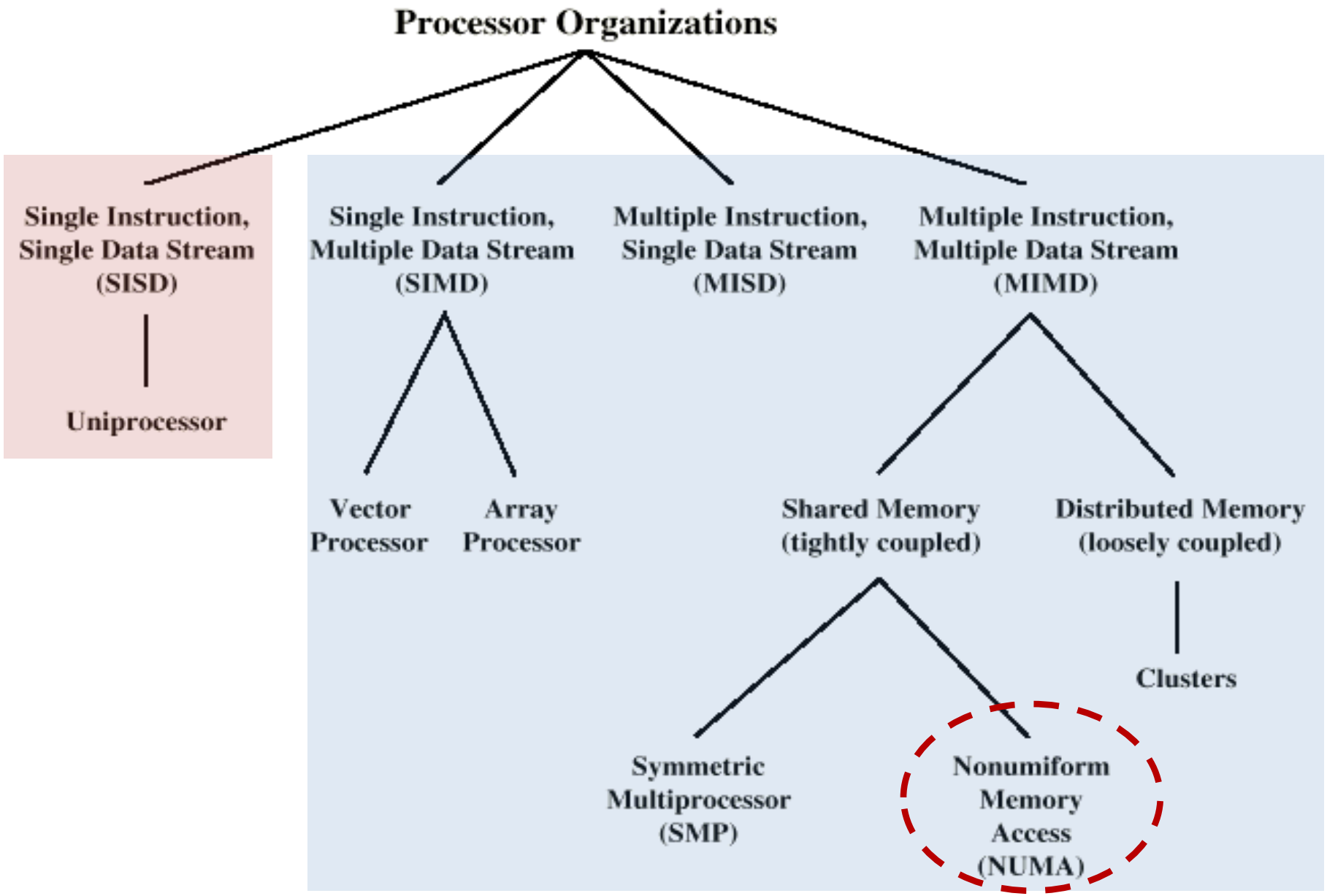
- **Performance**
  - **work** can be **done** in **parallel**
- **Availability**
  - **Failure** of a **single processor** **does not halt system**
- **Incremental growth**
  - **Adding** additional **processors** **enhances performance**
- **Scaling**
  - **Range of products** based on **number of processors**

Multithreading and Chip Multiprocessors

- **Instruction stream** divided into **smaller streams called "threads"**

- **Executed** in **parallel**

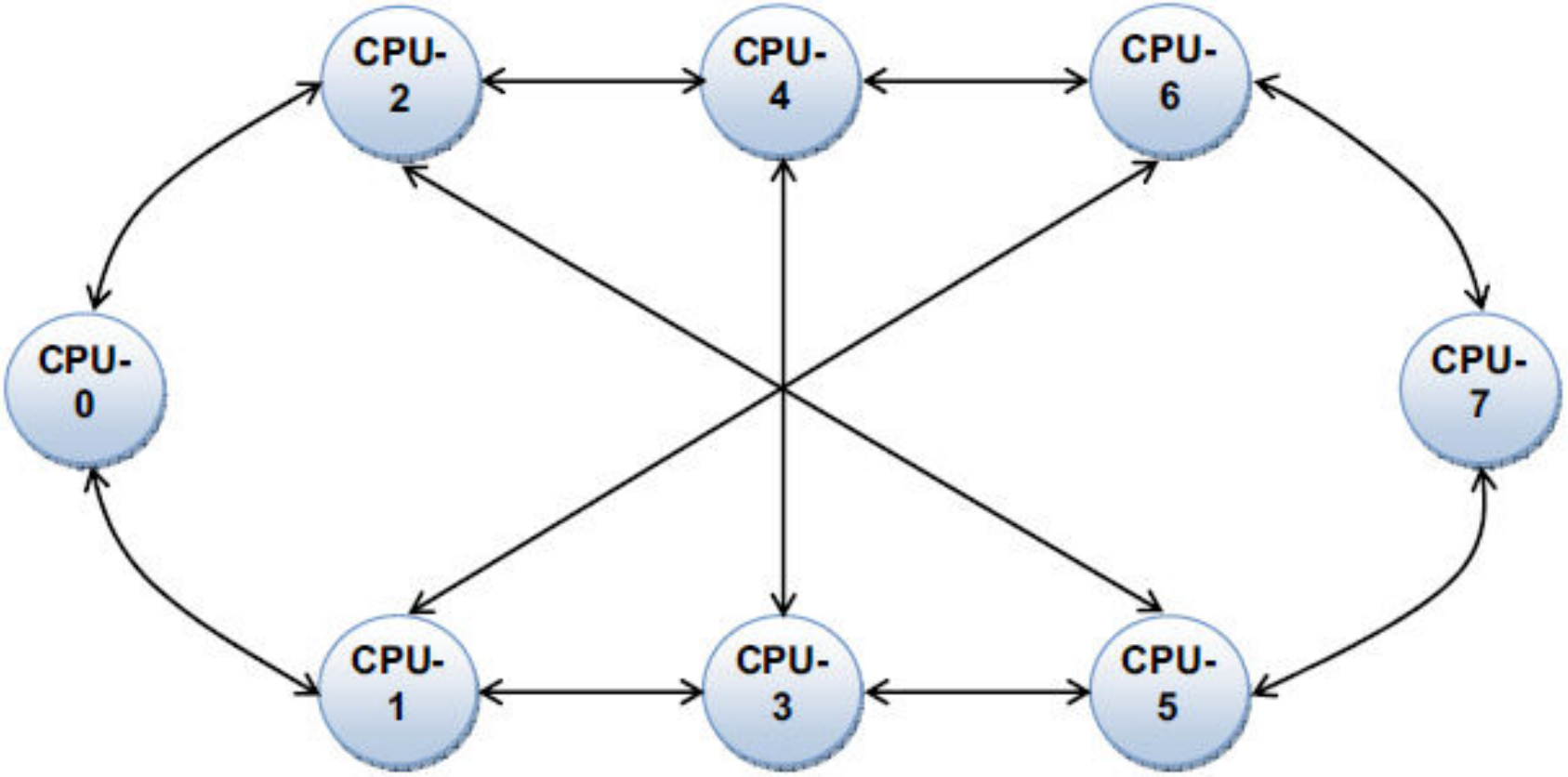
Taxonomy of Processor Architectures



Tightly Coupled - NUMA

- **Non-Uniform Memory Access (NUMA)**
  - Access times to different regions of memory **differs**

SunFire X4600M2 NUMA machine



Non-uniform Memory Access (NUMA)

- **Non-uniform memory access**
  - All **processors** have **access** to **all parts of memory**
  - **Access time** of processor **differs** depending on **memory region**
  - **Different processors** access **different regions** of memory at **different speeds**

- **Cache-coherent NUMA (cc-NUMA)**
  - **Cache coherence** is **maintained** among the **caches** of the **various processors**

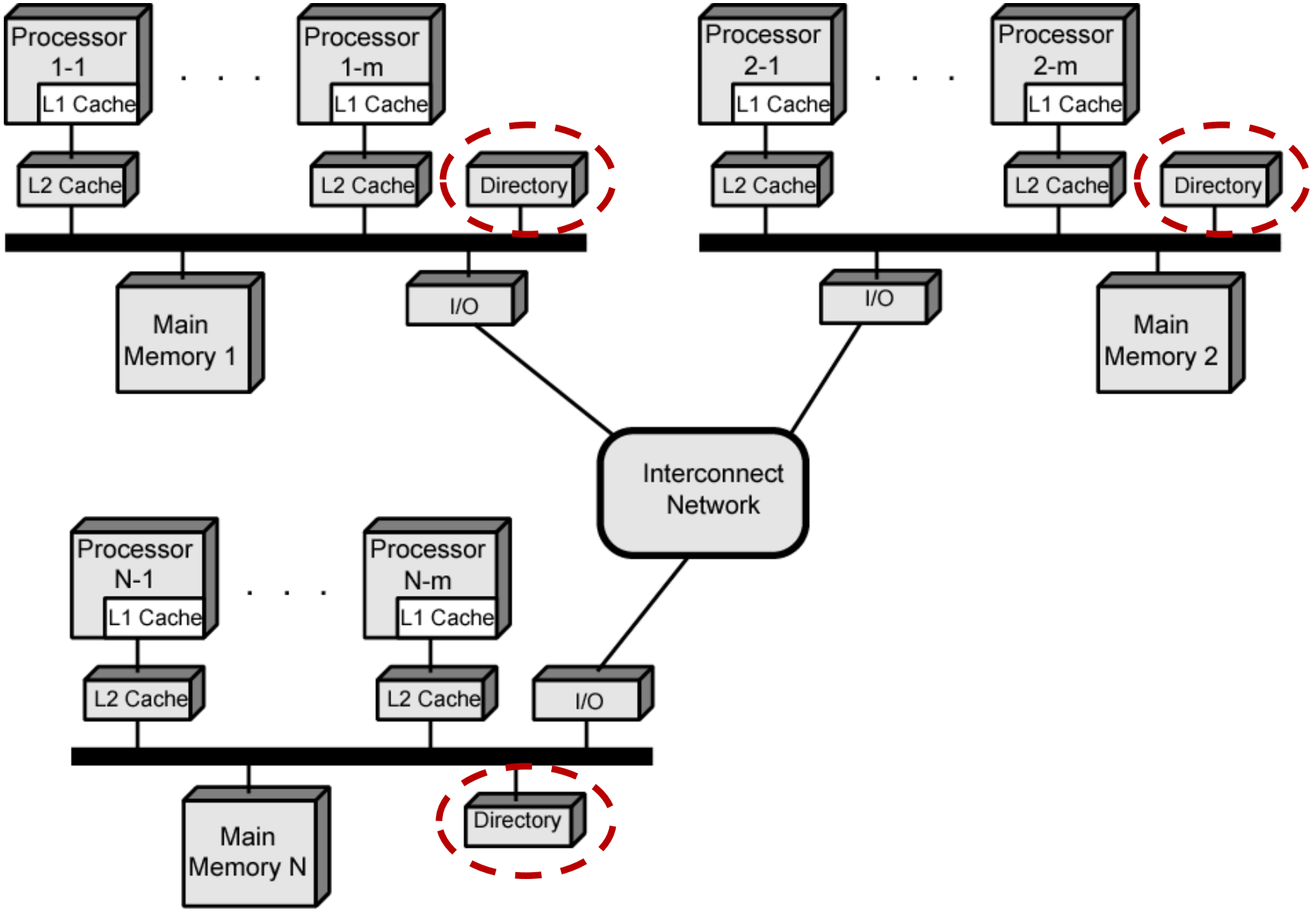
Motivation (Why NUMA)

- **SMP** has **practical limit** to **number** of **processors**
  - Bus traffic limits to between 16 and 64 processors

- In **clusters** each **node** has **own memory**:
  - **Apps do not see large global memory**
  - **Coherence maintained by software not hardware**

- **NUMA retains SMP flavour** while giving **large scale multiprocessing**

CC-NUMA Organization



CC-NUMA Operation

- **Each processor** has own **L1** and **L2** cache
- Each **node** has **own main memory**
- **Nodes connected** by some **networking facility**
- Each **processor** sees **single addressable memory**
- **Hardware support** for **read/write** to **non-local memories**, ***cache coherency***

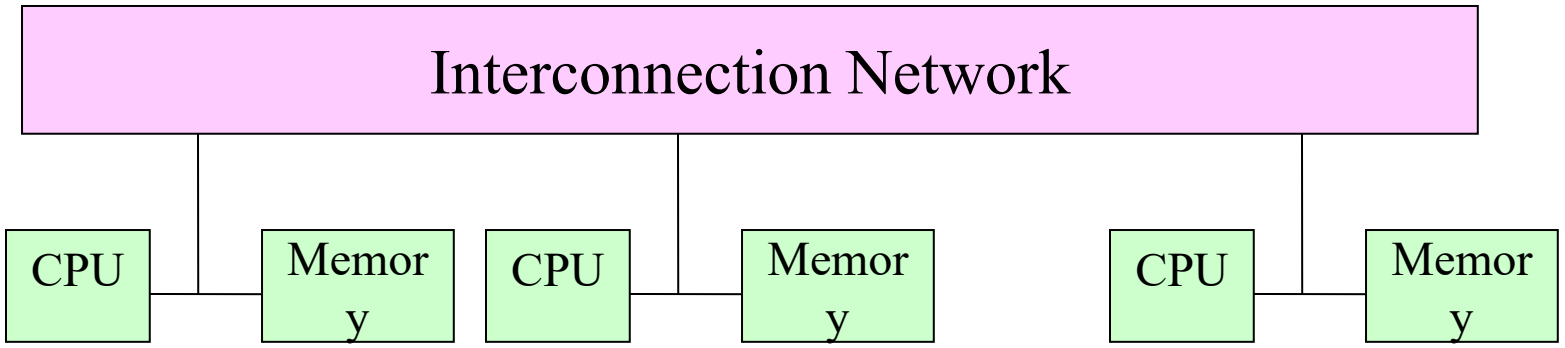
- **Memory request order:**
  1. **L1 cache** □ **L2 cache** (**local** to **processor**)
  2. Main memory (**local** to **node**)
  3. Remote memory

NUMA Pros & Cons

- Effective performance at higher levels of parallelism than SMP
- No major software changes
- Performance can breakdown if too much access to remote memory

Distributed Memory / Message Passing

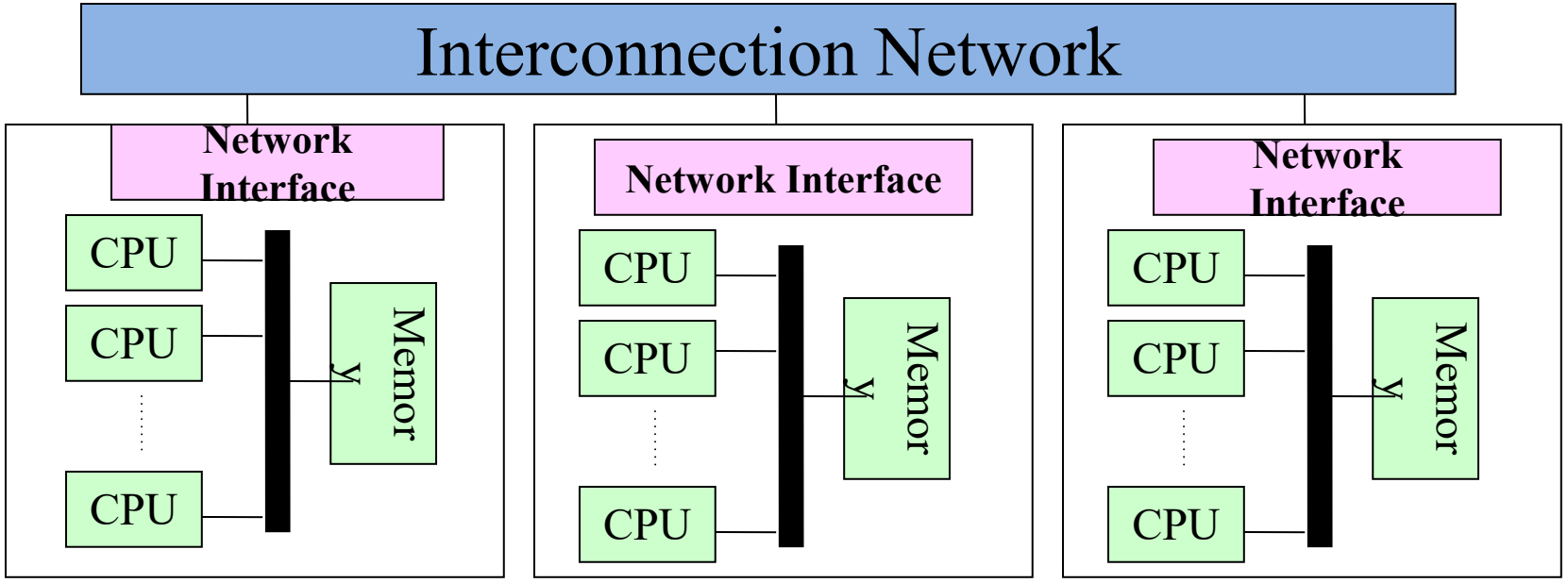
- Each **processor** has **access** to its **own memory only**
- **Data transfer** between **processors** is **explicit** (via message passing functions): **E.g., MPI library**
- User has **complete control/responsibility** for **data placement** and **management**



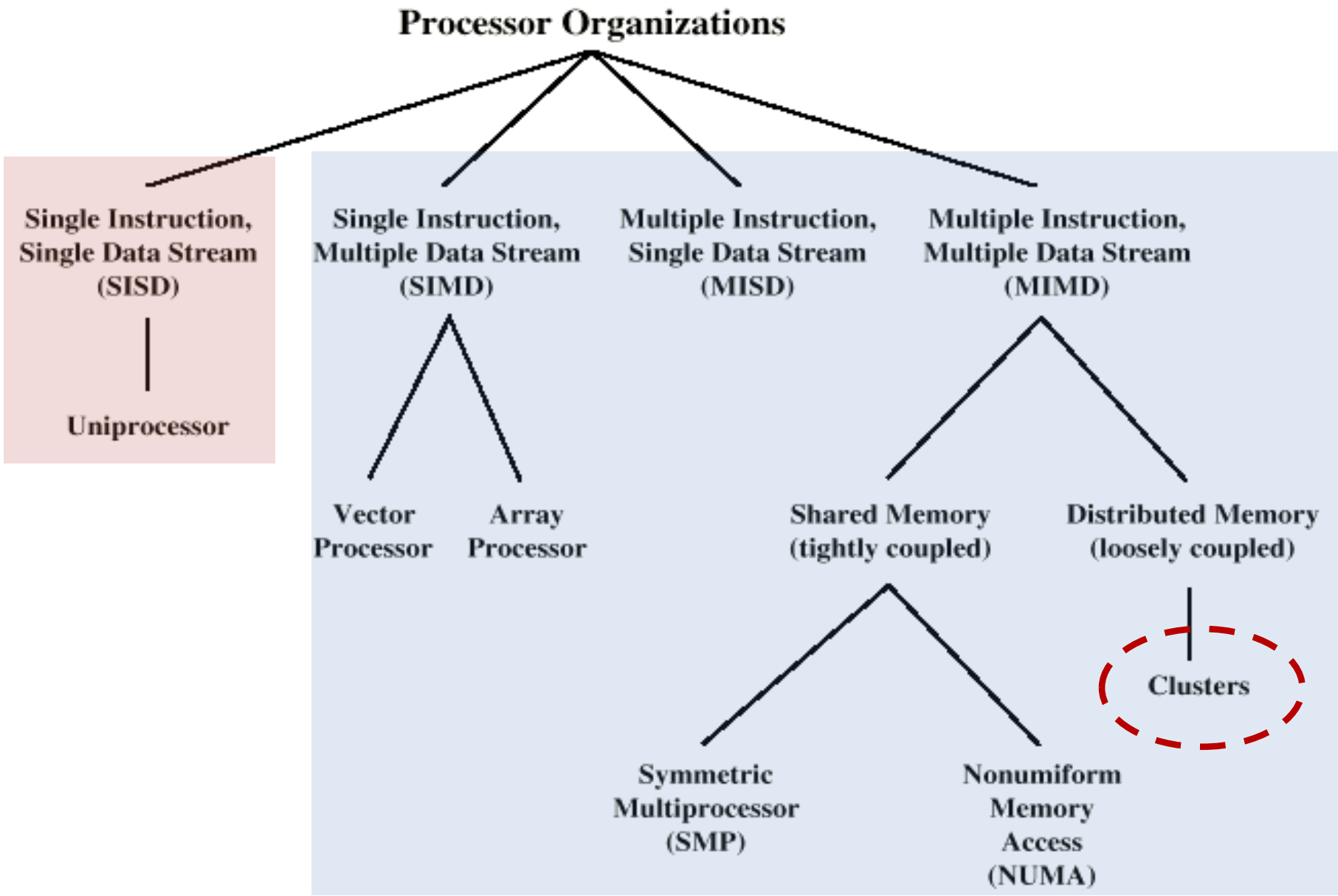
Hybrid Systems

- **Distributed memory system** with **multiprocessor shared memory nodes**

- **Most common parallel architecture**



Taxonomy of Processor Architectures



Distributed Computing

- We will discuss it at a later stage.

- Using **distributed systems** to **solve large problems**

- **Paradigms:**
  - **Cluster** computing
  - **Grid** computing
  - **Cloud** computing

Any Questions?